

```

1. Trieda SprávcaKompaktnejPamäte<TypBloku>
2.   rozširuje SprávcaPamäte<TypBloku> {
3.   konštruktor()
4.   konštruktor(veľkosť: int)
5.   konštruktor(iný: ↑SprávcaKompaktnejPamäte<TypBloku>)
6.   deštruktor()
7.   operácia prirad(
8.     iný: ↑SprávcaKompaktnejPamäte<TypBloku>
9.   ): ↑SprávcaKompaktnejPamäte<TypBloku>
10.  operácia porovnaj(
11.    iný: ↑SprávcaKompaktnejPamäte<TypBloku>
12.  ): bool
13.  operácia vymaž()
14.  operácia prideliPamäť(): ↑TypBloku
15.  operácia prideliPamäťNa(index: int): ↑TypBloku
16.  operácia uvoľniPamäť(ukazovateľ: ↑TypBloku)
17.  operácia uvoľniPamäťNa(index: int)
18.  operácia uvoľniPamäť()
19.  operácia dajVeľkosťAlokovanýchBlokov(): int
20.  operácia dajVeľkosťAlokovanejKapacity(): int
21.  operácia dajKapacitu(): int
22.  operácia zmeňVeľkosť(nováVeľkosť: int)
23.  operácia zmenšiPamäť()
24.  operácia vypočítajAdresu(dáta: ↑TypBloku): adresa
25.  operácia vypočítajPoradie(dáta: ↑TypBloku): int
26.  operácia dajBlokPamäte(index: int): ↑TypBloku
27.  operácia vymeň(index1: int, index2: int)
28.  vlastnosť báza: ↑TypBloku
29.  vlastnosť koniec: ↑TypBloku
30.  vlastnosť limit: ↑TypBloku
31.  konštanta VEĽKOSŤ_INIT: int ← 4
32. }

```

```

1. konštruktor SprávcaKompaktnejPamäte<TypBloku>() {
2.   inicializuj SprávcaKompaktnejPamäte<TypBloku>(VEĽKOSŤ_INIT)
3. }

```

```

1. konštruktor SprávcaKompaktnejPamäte<TypBloku>(veľkosť: int) {
2.   inicializuj predka SprávcaPamäte<TypBloku>()
3.   báza ← alokujPamäť(veľkosť, |TypBloku|)
4.   koniec ← báza
5.   limit ← báza + veľkosť
6. }

```

```

1. konštruktor SprávcaKompaktnejPamäte<TypBloku>(
2.   iný: ↑SprávcaKompaktnejPamäte<TypBloku>
3. ) {
4.   inicializuj SprávcaKompaktnejPamäte<TypBloku>(
5.     iný→dajPočetAlokovaných()
6.   )
7.   prirad(iný)
8. }

```

```

1. deštruktor SprávcaKompaktnejPamäte<TypBloku>() {
2.   uvoľniPamäť(báza)
3.   vráťPamäť(báza)
4.   báza ← NULL
5.   koniec ← NULL
6.   limit ← NULL
7.   finalizuj predka SprávcaPamäte<TypBloku>
8. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.prideIPamäť(
   ): ↑TypBloku {
2.   Vráť prideIPamäťNa(koniec - báza)
3. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.prideIPamäťNa(
   index: int
   ): ↑TypBloku {
2.   Ak (koniec = limit) potom {
3.     zmenVeľkosť(2 * početAlokovanýchBlokov)
4.   }
5.   Ak (koniec - báza > index) potom {
6.     presuňPamäť(báza + index + 1,
                  báza + index,
                  ((koniec - báza) - index) * |TypBloku|)
7.   }
8.   početAlokovanýchBlokov++
9.   koniec++
10.  Vráť vytvor TypBloku() na (báza + index)
11. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.uvoľniPamäť(
   ukazovateľ: ↑TypBloku
   ) {
2.   definuj premennú p: ↑TypBloku ← ukazovateľ
3.   Pokiaľ (p ≠ koniec) opakuje {
4.     finalizuj p
5.     p++
6.   }
7.   koniec ← ukazovateľ
8.   početAlokovanýchBlokov ← koniec - báza
9. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.uvoľniPamäťNa(
   index: int
   ) {
2.   finalizuj dajBlokPamäte(index)
3.   presuňPamäť(báza + index,
                báza + index + 1,
                (koniec - báza - index - 1) * |TypBloku|)
4.   koniec--
5.   početAlokovanýchBlokov--
6. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.uvoľniPamäť() {
2.   uvoľniPamäť(koniec - 1)
3. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.
   dajVeľkosťAlokovanýchBlokov(
   ): int {
2.   Vráť (koniec - báza) * |TypBloku|
3. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.
   dajVeľkosťAlokovanejKapacity(
   ): int {
2.   Vráť (limit - báza) * |TypBloku|
3. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.dajKapacitu(): int {
2.   Vráť limit - báza
3. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.prirad(
   iný: ↑SprávcaKompaktnejPamäte<TypBloku>
   ): ↑SprávcaKompaktnejPamäte<TypBloku> {
2.   Ak (self ≠ dajAdresu(iný)) potom {
3.     uvoľniPamäť(báza)
4.     početAlokovanýchBlokov ← iný→početAlokovanýchBlokov
5.     definuj premennú nováBáza: adresa ←
       zmeňVeľkosťPamäte(
         báza,
         iný→dajVeľkosťAlokovanejKapacity())
6.     Ak (nováBáza = NULL) potom {
7.       CHYBA
8.     }
9.     báza ← nováBáza
10.    koniec ← báza + početAlokovanýchBlokov
11.    limit ← báza + (iný→limit - iný→báza)
12.    Opakuj pre premennú i: int
       od 0 do početAlokovanýchBlokov - 1 {
13.      vytvor TypBloku((iný→báza + i)↓) na (báza + i)
14.    }
15.    Vráť self↓
16. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.zmeňVeľkosť(
    nováVeľkosť: int
) {
2.   Ak (nováVeľkosť = dajKapacitu()) potom {
3.     Vráť
4.   }
5.   Ak (nováVeľkosť < dajPočetAlokovanych()) potom {
6.     uvoľniPamäť(báza + nováVeľkosť)
7.   }
8.   definuj premennú nováBáza: adresa ←
        zmeňVeľkosťPamäte(báza, nováVeľkosť * |TypBloku|)
9.   Ak (nováBáza = NULL) potom {
10.    CHYBA
11.  }
12.  báza ← nováBáza
13.  koniec ← báza + početAlokovanychBlokov
14.  limit ← báza + nováVeľkosť
15. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.zmenšiPamäť() {
2.   definuj premennú nováVeľkosť: int ← koniec - báza
3.   Ak (nováVeľkosť < VEĽKOSŤ_INIT) potom {
4.     nováVeľkosť ← VEĽKOSŤ_INIT
5.   }
6.   zmeňVeľkosť(nováVeľkosť)
7. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.vymaž() {
2.   uvoľniPamäť(báza)
3. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.porovnaj(
    iný: ↑SprávcaKompaktnejPamäte<TypBloku>
): bool {
2.   Vráť (self = dajAdresu(iný)) ∨
        (početAlokovanychBlokov = iný→početAlokovanychBlokov ∧
         porovnajPamäť(báza,
            iný→báza,
            dajVeľkosťAlokovanychBlokov()) = 0)
3. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.vypočítajAdresu(
    dáta: ↑TypBloku
): adresa {
2.   definuj premennú p: ↑TypBloku ← báza
3.   Pokiaľ (p ≠ koniec ∧ p ≠ dajAdresu(dáta)) opakuj {
4.     p++
5.   }
6.   Keď platí (p = koniec)
      tak vráť NULL
      inak vráť p
7. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.vypočítajPoradie(
    dáta: ↑TypBloku
): int {
2.   Keď platí (dajAdresu(dáta) < koniec ∧ dajAdresu(dáta) ≥ báza)
      tak vráť dajAdresu(dáta) - báza
      inak vráť NEPLATNÝ_INDEX
3. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.dajBlokPamäte(
    index: int
): ↑TypBloku {
2.   Vráť (báza + index)↓
3. }

```

```

1. operácia SprávcaKompaktnejPamäte<TypBloku>.vymeň(
    index1: int,
    index2: int
) {
2.   vymeň(dajBlokPamäte(index1), dajBlokPamäte(index2))
3. }

```